

Flow Lab 网页端使用说明

实际页面截图与红色箭头讲解

Group 12 Backend

Flow Lab 总览：一页看完整个后端流程

左边输入，中心看模块流转，右边看 JSON；下方保留通信记录

The image shows the Flow Lab Backend Workflow Inspector interface. It is divided into several sections: a left sidebar for navigation, a top status bar, a central module board, and a right JSON inspector. Five numbered callouts highlight key features:

- 快捷操作栏** (Quick Action Bar): Located at the top right, it includes buttons for '检查后端' (Check Backend), '加载地点' (Load Location), '运行当前流程' (Run Current Flow), '导出日志' (Export Log), and '清空日志' (Clear Log). A red arrow points from this callout to the '检查后端' button.
- 输入区** (Input Area): Located on the left, it contains a '用户发言' (User Input) section with a text area and a '当前位置 / 起点' (Current Location / Start) section with a dropdown menu. A red arrow points from this callout to the '用户发言' text area.
- 模块板** (Module Board): Located in the center, it displays a grid of modules including 'Terminal', 'FastAPI', 'VLM Adapter', 'VisualLocalizer', 'Repository', 'Navigator', 'Exhibit Context', and 'Response'. A red arrow points from this callout to the 'VisualLocalizer' module.
- JSON 检查器** (JSON Inspector): Located on the right, it displays the JSON response from the API. A red arrow points from this callout to the JSON response area.
- 通信记录** (Communication Record): Located at the bottom, it displays a log of communication records. A red arrow points from this callout to the log area.

第一步：输入用户文本和基础参数

这里模拟终端发来的语音文本和导航条件；Codex 可以直接根据右侧 JSON 复现问题

BACKEND WORKFLOW INSPECTOR

后端调用流程可视化测试台

检查后端

加载地点

运行当前流程

导出日志

TERMINAL INPUT

文本、照片和任务参数

用户发言

例如：我现在在413A附近，想去429B，帮我规划路线。

当前位置 / 起点

目的地

LB-4F-ROOM-4C

LB-4F-ROOM-4C

根据提示

导航语言

4

中文

☐ 只使用无障碍路线

识别文字，每行一个

413A
西北侧楼梯

识别物体，每行一个，可写 label:confidence

413A:0.98
消防栓:0.7

场景描述

MODULE BOARD

模块调用和数据

Terminal

文本、图片、用户请求

VLM Adapter

图片转结构化视觉

vls.py

Repository

JSON 数据和路径

124 nodes

Exhibit Conte

展品 facts 和回答

exhibits.json

定位状态

暂无

用户发言

写自然语言测试句，例如“我在 413A 附近，想去 429B”。
点击“文本转 JSON”后会生成 intent。

起点、目的地、楼层和语言

这些字段会直接进入 `/api/v1/route`。
可测试中文、英文和无障碍路线。

视觉证据

识别文字和物体会组装成 `/api/v1/localize/visual`。
适合不用真实 VLM 时调试定位链路。

推荐协作方式

把一次失败用例的发言、视觉 JSON 和响应日志发给 Codex。
能快速定位是数据、VLM 还是路由问题。

第二步：选择要发送到哪个后端模块

不同按钮对应不同 API；可以单步测试，也可以顶部点击“运行当前流程”串起来

识别物体，每行一个，可写 label:confidence

413A:0.98
消防栓:0.7

场景描述

例如：走廊右侧可见房间号和消防栓

放置照片或点击选择
支持 JPEG / PNG / WebP，后端限制 5 MiB

展品 ID

ROBOT-001

展品问题

这个展品在哪里

文本转 JSON

发送视觉 JSON

发送图片

规划路线

展品上下文

展品 facts 和回答
exhibits.json

定位状态
暂无

COMMUNICATION HISTORY

所有通信记录

2026/7/14 21:39:47
15 ms

照片上传区

拖入照片或点击选择，发送图片时会走 `/api/v1/localize/image`。
这个接口需要配置 VLM API Key。

文本转 JSON / 发送视觉 JSON

不接终端、不接真实 VLM 时，
用这两个按钮模拟“语音理解”和“视觉定位”。

发送图片 / 规划路线

图片定位得到当前位置后，再用起点和目的地
调用 `/api/v1/route` 生成导航步骤。

展品上下文

调用 `/api/v1/exhibits/{id}/context`，
查看后端提供给讲解/RAG 的 facts。

第三步：看请求经过了哪些模块

模块变绿表示该模块刚参与过一次请求；右侧 JSON 用来核对真实请求和响应

The screenshot displays the application's interface, divided into two main sections: the Module Board on the left and the JSON Inspector on the right.

Module Board: Titled "MODULE BOARD" and "模块调用和数据流", it shows a grid of modules. The "current" status is "未定位". The modules are numbered 1 through 8:

- Terminal (1):** 文本、图片、用户目标; request
- FastAPI (2):** 请求校验和路由分发; 200
- VLM Adapter (3):** 图片转结构化视觉证据; vlm.py
- VisualLocalizer (4):** 视觉地标评分定位; visual_localization.py
- Repository (5):** JSON 数据和别名索引; 124 nodes
- Navigator (6):** Dijkstra 路线规划; navigation.py
- Exhibit Context (7):** 展品 facts 和回答约束; exhibits.json
- Response (8):** 返回给终端或 Codex 的结果; 200

At the bottom of the Module Board, there are three status indicators:

- 定位状态: 暂无
- 路线结果: 暂无
- 最后接口: /api/v1/places

JSON Inspector: Titled "JSON INSPECTOR" and "当前通信内容", it shows a "发送 JSON" button and a "响应 JSON" button. Below these is a large dark area displaying the JSON content. Red arrows point from the "发送 JSON" button to the "JSON 标签" callout and from the "响应 JSON" button to the "通信内容" callout. Another red arrow points from the "Response" module in the Module Board to the "结果摘要" callout.

模块板

定位看 VisualLocalizer；路线看 Navigator。
数据读取看 Repository。

JSON 标签

“发送 JSON”看请求体，“响应 JSON”看返回。
“路线步骤”看分段导航。

通信内容

这里是排错核心：把这段 JSON 发给队友
或 Codex，就能复现同一个请求。

结果摘要

底部会显示定位状态、路线距离/步数
和最后一次调用的接口。

第四步：用通信记录和 Codex 协作排错

每条记录都保存请求模块、接口、HTTP 状态、耗时、发送 JSON 和响应 JSON

ROBOT-001

这个展品在哪里

文本转 JSON

发送视觉 JSON

发送图片

规划路线

展品上下文

COMMUNICATION HISTORY

所有通信记录

2026/7/14 21:39:47	Browser -> Repository	200
15 ms	GET /api/v2/places	
2026/7/14 21:39:47	Browser -> FastAPI	200
13 ms	GET /health	
2026/7/14 21:38:48	Browser -> Repository	200
20 ms	GET /api/v2/places	
2026/7/14 21:38:48	Browser -> FastAPI	200
3 ms	GET /health	
2026/7/14 21:38:39	Browser -> Repository	200
14 ms	GET /api/v2/places	
2026/7/14 21:38:39	Browser -> FastAPI	200
13 ms	GET /health	

记录数量

确认本次实验产生了多少条通信。

请求流向

例如 Browser -> Repository，说明这次请求访问了数据仓库。

状态和耗时

200 表示成功；404、422、502、503 可直接定位错误类别。

点击回放

点任意日志，右侧 JSON 检查器会恢复当时的请求和响应。